Επομένως, αν επιθυμούμε να ταξινομήσουμε έναν πίνακα ακέραιων t, αρκεί η κλήση:

```
Arrays.sort(t);
```

Ίδια είναι η κλήση και για οποιονδήποτε άλλο θεμελιώδη τύπο, πλην του τύπου boolean.

6.2.2 Δυαδική αναζήτηση (Binary Search)

Κατά την αναζήτηση, ελέγχουμε έναν πίνακα για να διαπιστώσουμε αν περιλαμβάνει ή όχι μια τιμή. Αν η τιμή βρεθεί, επιστρέφεται η θέση της στον πίνακα. Αν η τιμή δεν βρεθεί, επιστρέφεται -1. Ειδικότερα, η δυαδική αναζήτηση εφαρμόζεται μόνο σε ταξινομημένους πίνακες. Η εφαρμογή της σε μη ταξινομημένους έχει απροσδιόριστα αποτελέσματα.
Η διεπαφή της συνάρτησης δυαδικής αναζήτησης για πίνακες ακέραιων έχει ως εξής:

```
public static int binarySearch(int[] a, int key)
```

Η binarySearch λαμβάνει ως παραμέτρους τον πίνακα στον οποίο επιθυμούμε να πραγματοποιήσουμε αναζήτηση και την τιμή που αναζητούμε. Επιστρέφει τη θέση στον πίνακα του στοιχείου που αναζητούμε ή -1 σε περίπτωση που το στοιχείο δεν υπάρχει στον πίνακα. Επομένως, αν έχουμε έναν μονοδιάστατο ταξινομημένο πίνακα ακέραιων, έστω t, και ψάχνουμε για το στοιχείο schElement, η κλήση θα πρέπει να είναι του τύπου:

```
int idx=Arrays.binarySearch(t, schElement);
```

6.2.3 Έλεγχος Ισότητας

Δύο πίνακες θεωρούνται ίσοι μεταξύ τους αν περιέχουν τα ίδια στοιχεία και με την ίδια σειρά.
Η διεπαφή της συνάρτησης ισότητας για πίνακες ακέραιων έχει ως εξής:

```
public static boolean equals(int[] a, int[] a2)
```

Επομένως, αν έχουμε τους πίνακες t1 και t2 και θέλουμε να τους συγκρίνουμε για ισότητα αρκεί η κλήση:

```
boolean tEquals=Arrays.equals(t1, t2);
```

6.2.4 Αρχικοποίηση

Η αρχικοποίηση πινάκων με την ίδια τιμή για κάθε στοιχείο του πίνακα γίνεται με τις συναρτήσεις fill. Η διεπαφή της fill για πίνακες ακέραιων έχει ως εξής:

```
public static void fill(int[] a, int val)
```

Επομένως, για να αρχικοποιήσουμε όλα τα στοιχεία ενός πίνακα ακέραιων, t, στην τιμή 1, αρκεί η κλήση

```
Arrays.fill(t,1);
```

6.2.5 Αναπαράσταση πίνακα ως String

Αν χρειαζόμαστε αναπαράσταση ενός πίνακα ως αλφαριθμητική σειρά μπορούμε να χρησιμοποιήσουμε την ομάδα συναρτήσεων toString. Η διεπαφή της συνάρτησης για πίνακες ακέραιων έχει ως εξής:

```
public static String toString(int[] a)
```

Για παράδειγμα, αν θέλουμε να τυπώσουμε τον πίνακα ακέραιων t, αρκεί ο ακόλουθος κώδικας:

```
System.out.println(Arrays.toString(t));
```

6.3 Πολυδιάστατοι πίνακες

Όπως αναφέραμε στην αρχή αυτού του κεφαλαίου, η Java υποστηρίζει και πολυδιάστατους πίνακες. Θα παρουσιάσουμε εδώ κυρίως τους δισδιάστατους πίνακες ενώ θα δώσουμε και παραδείγματα χρήσης πινάκων με περισσότερες διαστάσεις. Πάντως, ό,τι ισχύει για τους δισδιάστατους πίνακες, ισχύει αναλογικά και για πίνακες περισσότερων διαστάσεων.
Ένας πίνακας δύο διαστάσεων ενός τύπου στην Java είναι ένας πίνακας κάθε στοιχείο του οποίου είναι ένας μονοδιάστατος πίνακας του ίδιου τύπου.

```
static void twoDimArray() { //Κώδικας 6.9
    int[][] store = {
        {1, 2, 3, 4},
        {5, 6, 7},
        {8, 9, 10, 11, 12}
    };
    System.out.println(Arrays.toString(store[0]));
    System.out.println(Arrays.toString(store[1]));
    System.out.println(Arrays.toString(store[2]));
}
```

Κώδικας 6.9 *Παράδειγμα δήλωσης και αρχικοποίησης πίνακα δύο διαστάσεων με αρχικοποιητή*

Στο παράδειγμα του κώδικα 6.9 δηλώνουμε και αρχικοποιούμε τον δισδιάστατο πίνακα ακέραιων store. Ο πίνακας έχει δηλωθεί ως δισδιάστατος, καθώς τη δήλωση του τύπου int ακολουθούν δύο σύνολα από τετραγωνικές παρενθέσεις. Αντίστοιχα, μπορούμε να δηλώσουμε τρισδιάστατους πίνακες χρησιμοποιώντας τρία σύνολα τετραγωνικών παρενθέσεων, τετραδιάστατους με τέσσερα σύνολα, κοκ, χωρίς άνω όριο διαστάσεων. Στο συγκεκριμένο παράδειγμα, ο πίνακας store αρχικοποιήθηκε με τη χρήση αρχικοποιητή.
Τα στοιχεία του store είναι τρεις μονοδιάστατοι πίνακες ακέραιων. Επομένως, το μήκος του store είναι τρία, store.length==3. Τα δε στοιχεία του είναι τα εξής: το store[0] είναι ο πίνακας {1, 2, 3, 4}, το store[1] είναι ο πίνακας {5, 6, 7} και το store[2] είναι ο πίνακας {8, 9, 10, 11, 12}. Προσέξτε πως κάθε ένας από τους πίνακες-στοιχεία του store έχει διαφορετικό μέγεθος.

```
static void createTwoDimWithNew() { //Κώδικας 6.10
    int[][] store = new int[3][];
    store[0] = new int[4];
    store[1] = new int[3];
    store[2] = new int[5];
    int data = 0;
    for (int i = 0; i < store.length; i++) {
        for (int j = 0; j < store[i].length; j++) {
            store[i][j] = 1 + data++;
        }
    }
    System.out.println(Arrays.toString(store[0]));
    System.out.println(Arrays.toString(store[1]));
    System.out.println(Arrays.toString(store[2]));
}
```

Κώδικας 6.10 *Παράδειγμα δήλωσης και αρχικοποίησης πίνακα δύο διαστάσεων με χρήση της new*

Στον κώδικα 6.10 δημιουργούμε έναν δισδιάστατο πίνακα με τη βοήθεια της new. Στην περίπτωση αυτή, η μία διάσταση, αυτή που καθορίζει τα στοιχεία του store, υποχρεωτικά δηλώνεται μέσω μιας σταθεράς. Η δεύτερη μπορεί να δηλωθεί ή να μην δηλωθεί. Αν δηλωθεί όμως, τότε υποχρεωτικά όλα τα στοιχεία-πίνακες του store

θα έχουν τη διάσταση που δηλώθηκε κατά την εκτέλεση της new. Στο παράδειγμά μας, δεν έχουμε δηλώσει τη δεύτερη διάσταση. Έτσι μπορούμε να δημιουργήσουμε στοιχεία-πίνακες του store ποικίλων μεγεθών. Στην περίπτωση αυτή, τα στοιχεία του store πρέπει να δημιουργηθούν ένα προς ένα με χρήση της new. Για τα στοιχεία αυτά δεν είναι δυνατή η χρήση αρχικοποιητή, καθώς ο αρχικοποιητής είναι αποδεκτός μόνο στη γραμμή δήλωσης ενός πίνακα.

Στη συνέχεια, στον κώδικα 6.10, προσπελαύνουμε ένα προς ένα τα τελικά στοιχεία του store με μια εμφωλευμένη for προκειμένου να τους καταχωρήσουμε τιμές. Προσέξτε πως η εσωτερική for πηγαίνει μέχρι store[i].length. Συχνά παρατηρείται σε κώδικες αρχαρίων να χρησιμοποιείται το μήκος του δισδιάστατου πίνακα ως άνω όριο της εσωτερικής for. Αυτό είναι προφανώς λανθασμένη πρακτική, καθώς είναι δυνατό ο δισδιάστατος πίνακας να έχει διαφορετικό μήκος από τα στοιχεία-πίνακες που περιέχει.

Αν όμως όλα τα στοιχεία-πίνακες ενός δισδιάστατου πίνακα έχουν την ίδια διάσταση, τότε αν την δηλώσουμε, τα στοιχεία-πίνακες δημιουργούνται αυτόματα χωρίς τη χρήση της new για καθένα από αυτά.

```
static void createArrayWithCommonLength() { //Κώδικας 6.11
    int[][] s = new int[3][3];
    int data = 0;
    for (int i = 0; i < s.length; i++) {
        for (int j = 0; j < s[i].length; j++) {
            s[i][j] = 1 + data++;
        }
    }
    System.out.println(Arrays.toString(s[0]));
    System.out.println(Arrays.toString(s[1]));
    System.out.println(Arrays.toString(s[2]));
}
```

Κώδικας 6.11 *Δημιουργία δισδιάστατου πίνακα με σταθερό μήκος στοιχείων*

Στον κώδικα 6.11 τα στοιχεία του δισδιάστατου πίνακα s έχουν όλα το ίδιο μήκος. Σε αυτήν την περίπτωση κατά τη δημιουργία του s δηλώνουμε και τη δεύτερη διάσταση, οπότε ο s και τα στοιχεία του δημιουργούνται χωρίς να απαιτείται η αρχικοποιητή χρήση της new για κάθε στοιχείο ξεχωριστά.

Θα θεωρήσει κανείς πως σε αυτήν την περίπτωση δεν είναι αναγκαίο η εσωτερική for να αναφέρεται στο μήκος του τρέχοντος στοιχείου. Μια και το μήκος κάθε στοιχείου είναι ίδιο με το μήκος του s μπορεί να χρησιμοποιηθεί το μήκος του s και στην εσωτερική for. Αυτή η προσέγγιση είναι λανθασμένη. Η κατάσταση του πίνακα ανεξάρτητα από το πώς δημιουργήθηκε δεν είναι σταθερή. Είναι πιθανό να έχουν μεσολαβήσει μεταβολές από τη δημιουργία του μέχρι την επεξεργασία του. Για παράδειγμα, μετά τη δημιουργία του s είναι δυνατό να τρέξει κώδικας της μορφής s[1]=new int[7];. Ο κώδικας αυτός μεταβάλλει τον πίνακα στη θέση 1 του s. Για αυτόν τον λόγο, σε όλες τις περιπτώσεις, θα πρέπει να θυμόμαστε πως το μήκος ενός πίνακα, αξιόπιστα, το γνωρίζει μόνο ο ίδιος ο πίνακας.

Σε ποιες περιπτώσεις μπορούμε να αξιοποιήσουμε έναν πίνακα τριών διαστάσεων; Εξαρτάται από τα δεδομένα του προβλήματος που έχουμε να κωδικοποιήσουμε. Έστω, ότι έχουμε δεδομένα σχετικά με τις ώρες απασχόλησης ενός εργαζόμενου. Ας υποθέσουμε πως τα δεδομένα μας είναι δομημένα σε τέσσερις χρονικές περιόδους κάθε μία από τις οποίες αποτελείται από τρεις μήνες, κάθε ένας από τους οποίους περιλαμβάνει είκοσι δύο εργάσιμες ημέρες. Για κάθε μία από αυτές τις ημέρες, ο εργαζόμενος έχει εργαστεί έναν ορισμένο αριθμό ωρών. Τέτοιου τύπου δεδομένα είναι πολύ βολικό να φορτωθούν σε έναν τρισδιάστατο πίνακα. Για παράδειγμα, έστω ο πίνακας

```
int[][][] timesheet=new int[4][3][22];
```

Η πρώτη διάσταση αφορά τις περιόδους, η δεύτερη τον μήνα μιας περιόδου και η τρίτη, τις εργάσιμες ημέρες ενός μήνα. Επομένως, αν θέλουμε να προσπελάσουμε πόσες ώρες εργάστηκε ο εργαζόμενος την έβδομη ημέρα του δεύτερου μήνα της πρώτης περιόδου αρκεί να προσπελάσουμε το στοιχείο timesheet[0][1][6].

6.4 Βασικές επεξεργασίες

Συχνά απαιτείται να κάνουμε κάποιες βασικές εργασίες με πίνακες όπως να υπολογίσουμε το άθροισμα των στοιχείων ενός πίνακα αριθμών, να εντοπίσουμε το μεγαλύτερο ή το μικρότερο στοιχείο ενός πίνακα, να

πραγματοποιήσουμε σειριακή αναζήτηση ψάχνοντας για μια συγκεκριμένη τιμή σε έναν πίνακα, να ανακατέψουμε τα στοιχεία ενός πίνακα, να γεμίσουμε έναν πίνακα με τυχαίες τιμές. Σε αυτήν την ενότητα, θα δούμε τους κώδικες που μπορούν να αξιοποιηθούν για την υλοποίηση των αναφερόμενων επεξεργασιών.

6.4.1 Σειριακή αναζήτηση

Η σειριακή αναζήτηση σε αντίθεση με τη δυαδική μπορεί να εφαρμοστεί σε αταξινόμητους πίνακες. Όπως φανερώνει το όνομά της, ελέγχει ένα προς ένα τα στοιχεία του πίνακα συγκρίνοντάς τα με το στοιχείο που ψάχνουμε.
Ο κώδικας της σειριακής αναζήτησης είναι σχετικά απλός:

```
    public static void sequentialSearch() { //Κώδικας 6.12
        final int[] store = {4, 7, 9, 13};
        int schElement = 7;
        int idx = -1;
        for (int i = 0; i < store.length; i++) {
            if (store[i] == schElement) {
                idx = i;
                break;
            }
        }
        if (idx >= 0) {
            System.out.println("Ο ακέραιος " + schElement + " βρέθηκε στην θέση "
+ idx);
        } else {
            System.out.println("Ο ακέραιος " + schElement + " δεν βρέθηκε");
        }
    }
```

Κώδικας 6.12 *Σειριακή αναζήτηση σε μονοδιάστατο πίνακα ακέραιων*

Στον κώδικα 6.12 ορίζουμε καταρχάς τον πίνακα ακέραιων store και το στοιχείο για το οποίο θα ψάξουμε, το schElement. Επίσης, ορίζουμε μια ακέραιη μεταβλητή idx που αρχικοποιούμε στο -1. Αν κατά την αναζήτηση στον πίνακα βρεθεί το schElement, τότε στην idx θα εκχωρήσουμε τη θέση του στον πίνακα, οπότε η idx θα αποκτήσει τιμή μεγαλύτερη ή ίση με το 0. Έτσι θα γνωρίζουμε πως το στοιχείο βρέθηκε αλλά και πού ακριβώς είναι τοποθετημένο. Αν με το τέλος της αναζήτησης το στοιχείο δεν έχει βρεθεί, η idx θα παραμείνει μικρότερη από το μηδέν σηματοδοτώντας την αποτυχία εύρεσης του στοιχείου. Όπως έχουμε ήδη δει, η σύμβαση σύμφωνα με την οποία επιστρέφεται -1 για ένα στοιχείο που δεν βρέθηκε σε ένα πίνακα χρησιμοποιείται και από τη συνάρτηση binarySearch της κλάσης Arrays που παρουσιάστηκε στην 6.2.2.

Αφού λοιπόν ορίσουμε την idx μπαίνουμε σε έναν βρόχο for με τη βοήθεια του οποίου ελέγχουμε ένα προς ένα τα στοιχεία του store συγκρίνοντάς τα με το schElement. Αν για κάποιο από αυτά επιβεβαιωθεί ισότητα, τότε το schElement βρέθηκε, οπότε ενημερώνεται η idx και διακόπτεται η επαναληπτική διαδικασία.

Σε μια τέτοια συνάρτηση θα πρέπει να προσέξουμε ιδιαίτερα τον τύπο του πίνακα. Το πρότυπο του κώδικα 6.12 είναι κατάλληλο για πίνακες θεμελιωδών τύπων, όχι όμως και για πίνακες μη θεμελιωδών τύπων όπως ο τύπος String. Στην περίπτωση αυτή ο τελεστής ισότητας == θα πρέπει να αντικατασταθεί με τη συνάρτηση equals του τύπου του πίνακα όπως δείχνει ο κώδικας 6.13:

```
    public static void sequentialSearchReference() { //Κώδικας 6.13
        final String[] store = {"John", "Mary", "Jim", "Kelly"};
        String schElement = new String("Jim");
        int idx = -1;
        for (int i = 0; i < store.length; i++) {
            if (store[i].equals(schElement)) {
                idx = i;
                break;
            }
        }
        if (idx >= 0) {
```

```
            System.out.println("Το όνομα " + schElement + " βρέθηκε στην θέση " +
idx);
        } else {
            System.out.println("Το όνομα " + schElement + " δεν βρέθηκε");
        }
    }
```

Κώδικας 6.13 *Σειριακή αναζήτηση σε πίνακα αλφαριθμητικών σειρών*

6.4.2 Μέγιστο ή ελάχιστο στοιχείο

Ένα άλλο πρόβλημα είναι η αναζήτηση του μέγιστου στοιχείου ενός πίνακα. Πρόκειται για απλή επεξεργασία. Παραθέτουμε τον κώδικα υλοποίησης και στη συνέχεια τον σχολιάζουμε:

```
    public static void max() { //Κώδικας 6.14
        int[] t = {2, 6, 1, 9, 15};
        int max = t[0];
        int idx = 0;
        for (int i = 1; i < t.length; i++) {
            if (t[i] > max) {
                max = t[i];
                idx = i;
            }
        }
        System.out.println("Το μεγαλύτερο στοιχείο είναι το " + max + " και
βρίσκεται στην θέση " + idx);
```

Κώδικας 6.14 *Εύρεση του μεγαλύτερου στοιχείου μονοδιάστατου πίνακα*

Στον κώδικα 6.14 καταρχάς ορίζουμε τον πίνακα t. Στη συνέχεια, δύο βοηθητικές μεταβλητές, την max και την idx. Η max αρχικοποιείται στην τιμή του στοιχείου στη θέση μηδέν του πίνακα. Επίσης, η idx αρχικοποιείται ώστε να δείχνει στη θέση μηδέν. Ακολούθως, προσπελαύνουμε τα στοιχεία του πίνακα ένα προς ένα και όποτε βρίσκουμε τιμή μεγαλύτερη από την τρέχουσα τιμή του max ενημερώνουμε την max και την idx. Στο τέλος της διαδικασίας, η max έχει τη μεγαλύτερη τιμή του πίνακα και η idx τη θέση της.

Προσέξτε πως ο βρόχος for ξεκινάει από 1 και όχι από 0. Πράγματι, στο συγκεκριμένο πρόβλημα δεν υπάρχει λόγος να ξεκινήσει η σύγκριση από το 0. Αντίθετα, αν ξεκινούσε από το 0, κατά το πρώτο επαναληπτικό βήμα θα συγκρινόταν η max με το στοιχείο t[0]. Η max όμως σε αυτό το βήμα έχει τιμή ίση με το t[0], οπότε είναι περιττή η σύγκριση.

Στην περίπτωση που ο πίνακας έχει δύο στοιχεία ίσα μεταξύ τους και μεγαλύτερα από τα υπόλοιπα στοιχεία του πίνακα, ο κώδικας 6.14 εντοπίζει το στοιχείο στη μικρότερη θέση. Αν επιθυμούμε να βρούμε το στοιχείο στη μεγαλύτερη θέση αρκεί να τροποποιήσουμε τη συνθήκη της if σε t[i]>=max.

Αντίστοιχα, η εύρεση του μικρότερου στοιχείου, βρίσκεται αν αντικαταστήσουμε τη συνθήκη ελέγχου της if με την έκφραση t[i]<min ή t[i]<=min.

6.4.3 Άθροισμα

Ένα άλλο τυπικό πρόβλημα είναι ο υπολογισμός του αθροίσματος των στοιχείων ενός πίνακα. Πρόκειται επίσης για μια σειριακή προσπέλαση των στοιχείων του πίνακα και προσαύξηση ενός αθροιστή κατά την τιμή κάθε στοιχείου. Ο κώδικας 6.15 παρουσιάζει τη λύση.

```
    public static void sum() { //Κώδικας 6.15
        int[] t = {2, 6, 1, 9, 15};
        int sum = 0;
        for (int i = 0; i < t.length; i++) {
            sum += t[i];
        }
        System.out.println("Το άθροισμα είναι " + sum);
    }
```

Κώδικας 6.15 *Άθροισμα στοιχείων μονοδιάστατου πίνακα*

6.4.4 Ανακατανομή

Σε πολλές εφαρμογές απαιτείται τυχαία ανακατανομή των στοιχείων ενός πίνακα. Για κάθε στοιχείο του πίνακα, υπολογίζουμε μια τυχαία θέση μέσα στα όρια του πίνακα και αντιμεταθέτουμε το στοιχείο στην τρέχουσα θέση με το στοιχείο στην τυχαία θέση.

```
public static void shuffle() { //Κώδικας 6.16
    int[] t = {2, 6, 1, 9, 15};
    Arrays.sort(t);
    System.out.println(Arrays.toString(t));
    Random r = new Random();

    for (int i = 0; i < t.length; i++) {
        int rIdx = r.nextInt(t.length);
        int tmp = t[i];
        t[i] = t[rIdx];
        t[rIdx] = tmp;
    }
    System.out.println(Arrays.toString(t));
}
```

Κώδικας 6.16 *Ανακατανομή στοιχείων πίνακα*

Στον κώδικα 6.16 δημιουργούμε το πίνακα t, τον ταξινομούμε και τον τυπώνουμε έτσι ώστε να είναι εμφανής η διαφορά από τον ανακατεμένο πίνακα. Στη συνέχεια υλοποιείται ο αλγόριθμος ανακατανομής που περιγράφηκε αμέσως προηγουμένως.

6.4.5 Αρχικοποίηση με τυχαίες τιμές

Πολλές φορές χρειαζόμαστε να γεμίσουμε έναν πίνακα με τυχαίες τιμές είτε για λόγους ελέγχου είτε επειδή η εφαρμογή μας το απαιτεί. Στον κώδικα 6.17 ο πίνακας γεμίζει με τυχαίες τιμές από το 0 έως 99.

```
public static void fillRandom() { //Κώδικας 6.17
    int[] t = new int[20];
    Random r = new Random();
    for (int i = 0; i < t.length; i++) {
        t[i] = r.nextInt(100);
    }
    System.out.println(Arrays.toString(t));
}
```

Κώδικας 6.17 *Αρχικοποίηση πίνακα με τυχαίες τιμές*

6.4.6 Τελικοί πίνακες

Η δεσμευμένη λέξη final μπορεί να χρησιμοποιηθεί και με μεταβλητές που αναφέρονται σε πίνακες. Η σημασία της είναι ακριβώς ίδια με τη σημασία που έχει όταν χρησιμοποιείται με μεταβλητές τιμής. Ωστόσο, επειδή οι μεταβλητές αναφοράς έχουν διαφορετική συμπεριφορά, συχνά παρατηρείται σύγχυση στον αρχάριο προγραμματιστή σε σχέση με τις επιπτώσεις του χαρακτηρισμού ως final ενός πίνακα.

Όπως αναφέρουμε στην ενότητα 3.1.2, ο χαρακτηρισμός ως final μιας μεταβλητής έχει ως συνέπεια να επιτρέπεται εκχώρηση στη μεταβλητή μόνο μία φορά. Το ίδιο ακριβώς συμβαίνει και με τις μεταβλητές πινάκων. Επομένως, ο κώδικας που ακολουθεί παράγει λάθος χρόνου μεταγλώττισης.

```
final int[] t={1,3,5,7};
t=new int[9];
```

Ωστόσο, η μεταβολή των στοιχείων του πίνακα είναι διαφορετική υπόθεση. Αυτή μπορεί να γίνει χωρίς να μεταβληθεί η αναφορά στον πίνακα.

Επομένως, ο ακόλουθος κώδικας είναι απολύτως σωστός:

```
final int[] t={1,3,5,7};
t[0]=10;
```

6.5 Λυμένες Ασκήσεις

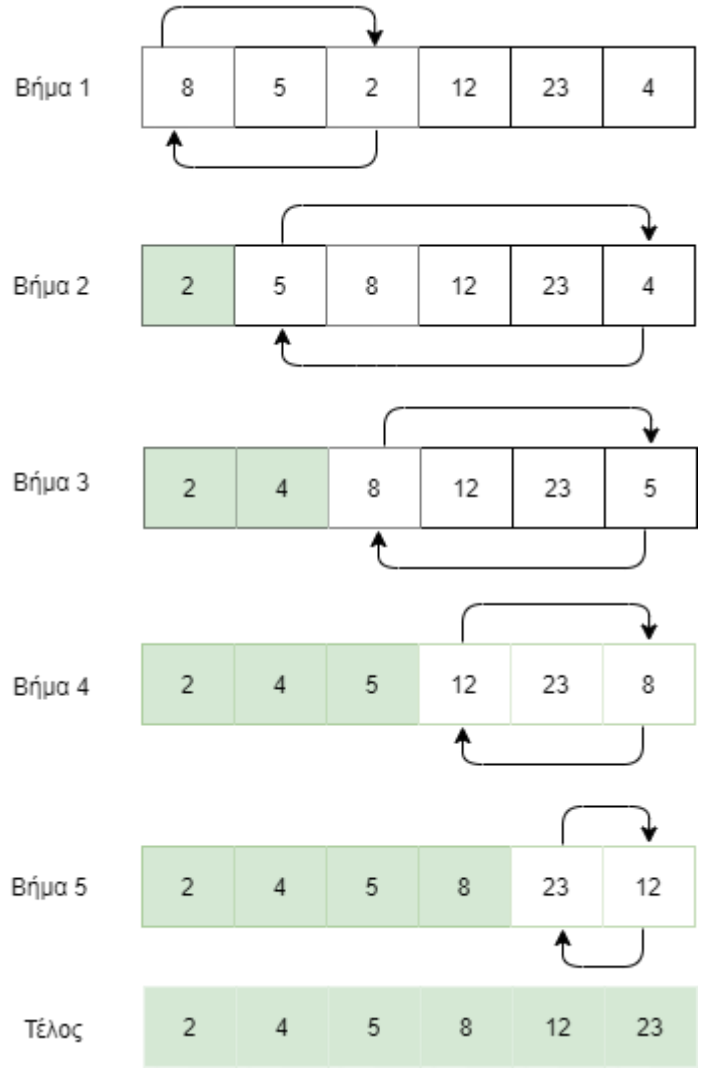
Παρουσιάζουμε εδώ μια σειρά από βασικές επεξεργασίες πινάκων.

6.5.1 Ταξινόμηση

Να αναπτύξετε συνάρτηση στην οποία να ορίσετε έναν πίνακα ακέραιων και στη συνέχεια να τον ταξινομήσετε κατ' αύξουσα σειρά χωρίς να χρησιμοποιήσετε συνάρτηση sort της Arrays.

Λύση

Υπάρχουν πολλοί αλγόριθμοι ταξινόμησης δεδομένων. Για τη λύση αυτής της άσκησης θα χρησιμοποιήσουμε ίσως τον απλούστερο όλων, γνωστό ως selection sort [1]. Το βασικό βήμα σε αυτήν την ταξινόμηση είναι η ανταλλαγή θέσης μεταξύ μικρότερου στοιχείου και του στοιχείου που είναι στην αρχή του αταξινόμητου μέρους του πίνακα. Το βήμα επαναλαμβάνεται από το πρώτο μέχρι το προτελευταίο στοιχείο του πίνακα. Με το τέλος της επανάληψης ο πίνακας είναι ταξινομημένος. Εξηγούμε λεπτομερέστερα με τη βοήθεια του σχήματος 6.2.



Σχήμα 6.2 *Αναπαράσταση του selection sort*

Βήμα 1: Ολόκληρος ο πίνακας είναι αταξινόμητος. Το πρώτο στοιχείο του αταξινόμητου πίνακα βρίσκεται στη θέση 0 και έχει τιμή 8. Το μικρότερο στοιχείο του πίνακα βρίσκεται στη θέση 2 και έχει τιμή 2. Τα στοιχεία θα ανταλλάξουν θέσεις.

Βήμα 2: Μετά την ανταλλαγή θέσεων του βήματος 1, το μικρότερο στοιχείο βρίσκεται στη θέση 0. Το τμήμα του πίνακα που περιλαμβάνει τη θέση 0 θεωρείται ταξινομημένο. Το μη ταξινομημένο τμήμα του πίνακα εκτείνεται από τη θέση 1 έως το τέλος του πίνακα. Επαναλαμβάνεται η λογική του βήματος 1 για το αταξινόμητο τμήμα του πίνακα. Το μικρότερο στοιχείο με τιμή 4 θα ανταλλάξει θέση με το στοιχείο με τιμή 5.

Βήμα 3: Με την ίδια λογική το 5 θα ανταλλάξει θέση με το 8.

Βήμα 4: Το 8 θα ανταλλάξει θέση με το 12

Βήμα 5: Το 12 θα ανταλλάξει θέση με το 23

Τέλος: Ο πίνακας είναι ταξινομημένος

Στην συνέχεια δίνουμε την υλοποίηση της selectionSort:

```
1   static void selectionSort(String[] args) { //Κώδικας 6.18
2       int[] tbl = {2, 1, 6, 3, 4, 9, 8, 5, 7, 4};
3       for (int i = 0; i < tbl.length - 1; i++) {
4           int min = tbl[i];
5           int idx = i;
6           for (int j = i + 1; j < tbl.length; j++) {
7               if (tbl[j] < min) {
8                   min = tbl[j];
9                   idx = j;
10              }
11          }
12
13          if (idx != i) {
14             int tmp = tbl[i];
15             tbl[i] = tbl[idx];
16             tbl[idx] = tmp;
17         }
18      }
19      System.out.println(Arrays.toString(tbl));
20  }
```

Κώδικας 6.18 *Υλοποίηση του selection sort*

Στη γραμμή 2 ορίζουμε έναν πίνακα ακέραιων. Στη γραμμή 3 ξεκινάμε τη βασική επαναληπτική διαδικασία. Στις γραμμές 4 έως 11 αναζητούμε το μικρότερο στοιχείο του πίνακα από τη θέση i και μετά. Αν το μικρότερο στοιχείο βρίσκεται σε θέση διαφορετική από το i, αντιμεταθέτουμε τα δύο στοιχεία (γραμμές 11 έως 15). Ο εξωτερικός βρόχος (γραμμή 3) ελέγχει μέχρι tbl.length-2. Αυτό είναι λογικό, καθώς το tbl.length-1 είναι η τελευταία θέση του πίνακα και δεν έχει νόημα να ελέγξουμε αν μετά από αυτήν υπάρχει στοιχείο με μικρότερη τιμή. Στη γραμμή 19 εμφανίζουμε τον ταξινομημένο πίνακα.

6.5.2 Δυαδική αναζήτηση

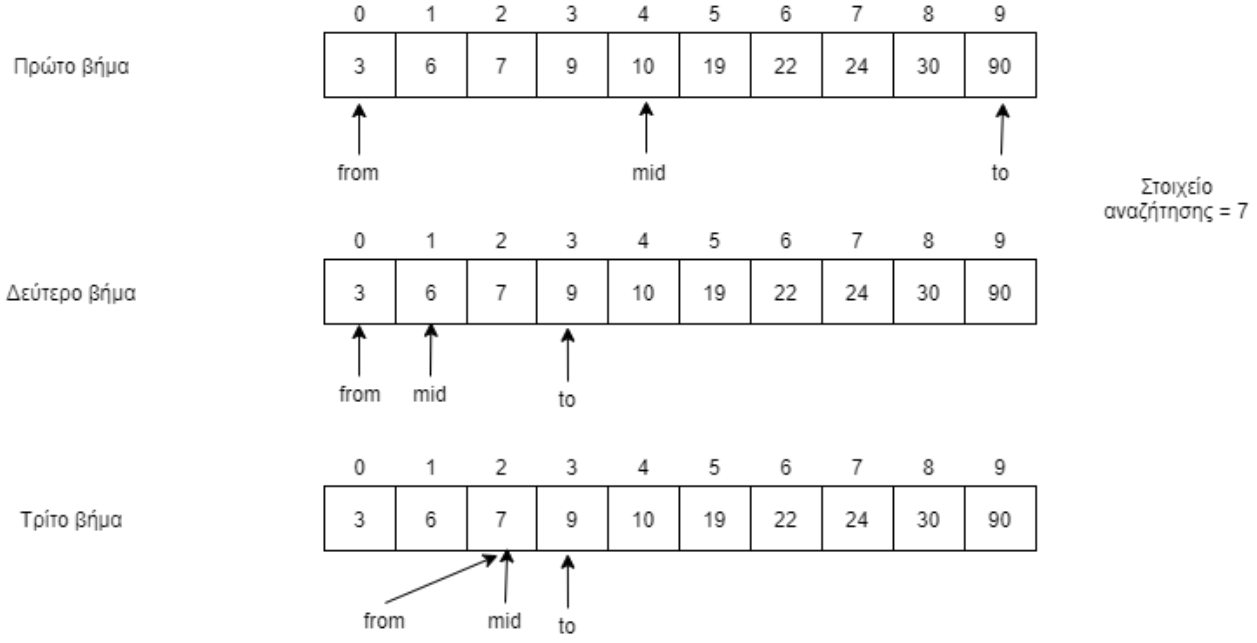
Να αναπτύξετε συνάρτηση Java στην οποία να ορίσετε έναν μονοδιάστατο πίνακα ακέραιων, έναν ακέραιο και στη συνέχεια να πραγματοποιήσετε δυαδική αναζήτηση στον πίνακα και να βρείτε τον ακέραιο χωρίς να χρησιμοποιήσετε τις συναρτήσεις δυαδικής αναζήτησης της Arrays.

Λύση

Η βασική ιδέα της δυαδικής αναζήτησης (binary search) [2] είναι να εξετάζουμε κάθε φορά ένα τμήμα του πίνακα για το οποίο ελέγχουμε αν η τιμή που αναζητούμε είναι μεγαλύτερη, μικρότερη ή ίση από την τιμή στο μέσο του τμήματος. Αν είναι ίση, το στοιχείο αναζήτησης βρέθηκε. Αν είναι μικρότερη, μας μένει να ψάξουμε μόνο στο πρώτο μισό μέρος του τμήματος του πίνακα. Αν είναι μεγαλύτερη, μας μένει να ψάξουμε μόνο στο

δεύτερο μισό μέρος του τμήματος του πίνακα. Με αυτόν τον τρόπο, η δυαδική αναζήτηση υποδιπλασιάζει σε κάθε επαναληπτικό βήμα τα στοιχεία του πίνακα που μένουν προς έλεγχο. Κατά το πρώτο βήμα, το τμήμα του πίνακα που έχουμε να ψάξουμε συμπίπτει με ολόκληρο τον πίνακα.

Για να υλοποιήσουμε τη δυαδική αναζήτηση χρειαζόμαστε πέραν του πίνακα και του στοιχείου αναζήτησης και δύο δείκτες, ας τους ονομάσουμε from και to, που καθορίζουν τα όρια του τμήματος που ελέγχουμε σε κάθε επαναληπτικό βήμα. Στο σχήμα 6.3 παρουσιάζεται παράδειγμα δυαδικής αναζήτησης:



Σχήμα 6.3 *Παράδειγμα δυαδικής αναζήτησης*

Στην αρχή της αναζήτησης, ο δείκτης from δείχνει στην αρχή του πίνακα και ο δείκτης to στο τέλος. Συμπεριλαμβάνεται έτσι όλος ο πίνακας στην αναζήτηση. Με βάση τις τιμές των from και to υπολογίζεται το μέσο, mid, του πίνακα. Στη συνέχεια, εφόσον το στοιχείο αναζήτησης είναι μεγαλύτερο από την τιμή στο μέσο του πίνακα και ο πίνακας είναι ταξινομημένος κατ' αύξουσα σειρά, εξάγεται το συμπέρασμα πως το στοιχείο αναζήτησης, αν υπάρχει, θα πρέπει να αναζητηθεί στο δεύτερο τμήμα του πίνακα, δηλαδή στο τμήμα που αρχίζει από mid+1 και το τέλος του συμπίπτει με το τέλος του πίνακα. Αντίθετα, αν όπως στο παράδειγμά μας, το στοιχείο αναζήτησης είναι μικρότερο από την τιμή στο μέσο του πίνακα, τότε η αναζήτηση θα πρέπει να συνεχίσει με το πρώτο μισό τμήμα του πίνακα, δηλαδή να εφαρμοστεί στο τμήμα from έως mid-1.

Έτσι στο δεύτερο βήμα, ο δείκτης to γίνεται ίσος με mid-1 και άρα το νέο μέσο τοποθετείται στη θέση 1 του πίνακα και δείχνει μια θέση μετά το μέσο του πίνακα. Στη συνέχεια, το στοιχείο αναζήτησης είναι μεγαλύτερο από την τιμή στη θέση 1, οπότε ο δείκτης from γίνεται mid+1. Το νέο μέσο βρίσκεται στη θέση 2 όπου βρίσκεται και το στοιχείο αναζήτησης. Τέλος, ελέγχεται το μέσο και βρίσκεται ίσο με το στοιχείο αναζήτησης, οπότε η αναζήτηση διακόπτεται.

Ο κώδικας 6.19 παρέχει την υλοποίηση σε Java του αλγόριθμου δυαδικής αναζήτησης.

```
public static void binarySearch() { //Κώδικας 6.19
    int[] tbl = new int[]{1, 3, 4, 5, 7, 9, 11, 23, 34, 50};
    int idx = -1;
    final int schElement = 24;
    int from = 0, to = tbl.length - 1, mid = (to - from) / 2;
    while (to >= from) {
        if (tbl[mid] == schElement) {
            idx = mid;
            break;
        } else if (schElement < tbl[mid]) {
            to = mid - 1;
        } else {
            from = mid + 1;
        }
```

```
            mid = from + (to - from) / 2;
        }
        if (idx == -1) {
            System.out.println("Element " + schElement + " was not found");
        } else {
            System.out.println("Element " + schElement + " found at position " +
idx);
        }
    }
```

Κώδικας 6.19 *Υλοποίηση δυαδικής αναζήτησης*

Στον κώδικα 6.19 καθορίζουμε καταρχάς τον πίνακα στον οποίο θα εφαρμόσουμε τη δυαδική αναζήτηση. Στη συνέχεια ορίζουμε τις απαραιτητες βοηθητικές μεταβλητές idx, from, to και mid. Η μεταβλητή from αρχικοποιείται στο 0 και η to στο tbl.length-1. Έτσι, η αναζήτηση αρχικά αφορά όλο τον πίνακα. Αμέσως μετά, καθορίζουμε το στοιχείο αναζήτησης, schElement. Στη συνέχεια, η ροή εισέρχεται σε έναν βρόχο while που ελέγχεται από τη συνθήκη to>=from. Όσο αυτή η συνθήκη είναι αληθής, υπάρχει τμήμα του πίνακα που δεν έχει ελεγχθεί. Στη συνέχεια, αν η τιμή στο μέσο του πίνακα είναι ίση με την τιμή που αναζητούμε, το στοιχείο βρέθηκε, οπότε ενημερώνεται η idx και διακόπτεται η while, αλλιώς, αν το στοιχείο αναζήτησης είναι μικρότερο από την τιμή στο μέσο του πίνακα, ενημερώνεται η μεταβλητή to ώστε η αναζήτηση να περιοριστεί στο πρώτο μισό του πίνακα. Αν όμως το στοιχείο αναζήτησης είναι μεγαλύτερο από την τιμή στο μέσο του πίνακα, ενημερώνεται η from ώστε η αναζήτηση να συνεχιστεί στο δεύτερο μισό του πίνακα. Στο επόμενο βήμα υπολογίζεται το νέο μέσο. Τέλος, έξω από τον βρόχο while ενημερώνουμε με τα αποτελέσματα της αναζήτησης με την εμφάνιση κατάλληλων μηνυμάτων.

6.5.3 Ισότητα Πινάκων

Να αναπτύξετε συνάρτηση στην οποία να ορίσετε τους ακόλουθους πίνακες:

```
String[] tbl1 = new String[]{new String("John"), "Maria",
"Kely"};
String[] tbl2 = {"John", "Kely", "Maria"};
```

Στη συνέχεια να ελέγξετε αν είναι ίσοι μεταξύ τους με βάση τον ακόλουθο ορισμό ισότητας: Δύο πίνακες από Strings είναι ίσοι μεταξύ τους, εφόσον έχουν το ίδιο μέγεθος και κάθε στοιχείο του ενός είναι και στοιχείο του άλλου ανεξάρτητα από τη θέση που έχουν τα στοιχεία στους πίνακες.
Προσέξτε πως αυτός ο ορισμός ισότητας πινάκων διαφέρει από τον ορισμό στον οποίο βασίζονται οι συναρτήσεις equals της Array.

Λύση

```
public static void stringEquality() { //Κώδικας 6.20
    String[] tbl1 = new String[]{new String("John"), "Maria", "Kely"};
    String[] tbl2 = {"John", "Kely", "Maria"};
    boolean equal = tbl1.length == tbl2.length;
    for (int i = 0; i < tbl1.length && equal; i++) {
        boolean found = false;
        for (String cName : tbl2) {
            if (tbl1[i].equals(cName)) {
                found = true;
                break;
            }
        }
        equal = found;
    }
    System.out.println(equal);
}
```

Κώδικας 6.20 *Έλεγχος ισότητας πινάκων*

Στον κώδικα 6.20 ορίζουμε αρχικά τους πίνακες όπως δίνονται στην εκφώνηση. Στη συνέχεια αρχικοποιούμε τη μεταβλητή equal στην τιμή της έκφρασης tbl1.length == tbl2.length. Αν οι πίνακες δεν έχουν ίδιο μήκος, η equal γίνεται false. Στη συνέχεια η ροή δεν μπαίνει στον βρόχο for και απλά εμφανίζει την τιμή false. Αν όμως τα μήκη των πινάκων είναι ίσα μεταξύ τους, τότε η ροή εισέρχεται στον εξωτερικό βρόχο for. Εκεί ελέγχεται κάθε στοιχείο του tbl1 αν υπάρχει και στον tbl2. Όσο ένα στοιχείο του tbl1 βρίσκεται και στον tbl2, η βοηθητική μεταβλητή found γίνεται true και αντίστοιχα ενημερώνεται και η equal. Αν ένα στοιχείο του tbl1 δεν βρεθεί στον tbl2, η found γίνεται false και διακόπτεται η εσωτερική for. Ακολούθως γίνεται false η equal η οποία θα διακόψει και την εξωτερική for.

6.5.4 Αντιστροφή πίνακα

Να γράψετε κώδικα ο οποίος με δεδομένο τον πίνακα int[] tbl = {1, 2, 3, 14, 5, 6, 7, 8, 9} παράγει έναν πίνακα που περιέχει τα ίδια στοιχεία με τον tbl σε αντεστραμμένη διάταξη, δηλαδή στη θέση 0 έχει το 9 και στη θέση 8 το 1.

Λύση

```
public static void reverse() { //Κώδικας 6.21
    int[] tbl = {1, 2, 3, 14, 5, 6, 7, 8, 9};
    int[] reversed = new int[tbl.length];
    for (int i = 0; i < tbl.length; i++) {
        reversed[tbl.length - 1 - i] = tbl[i];
    }
    System.out.print(Arrays.toString(reversed));
}
```

Κώδικας 6.21 *Παραγωγή πίνακα με αντεστραμμένη διάταξη στοιχείων*

Καταρχάς δημιουργούμε τον πίνακα reversed με ίδιο μήκος με τον tbl. Στη συνέχεια τοποθετούμε στη θέση tbl.length - 1 – i του reversed το στοιχείο που βρίσκεται στη θέση i του tbl.

6.5.5 Συνένωση πινάκων

Να γράψετε κώδικα που με δεδομένους τους πίνακες int[] t1={1,2,3,4,5}, t2={6,7,8,9} παράγει έναν πίνακα που συνενώνει τους t1 και t2.

Λύση

```
public static void merge() { //Κώδικας 6.22
    int[] t1 = {1, 2, 3, 4, 5}, t2 = {6, 7, 8, 9};
    int[] rVal = new int[t1.length + t2.length];
    System.arraycopy(t1, 0, rVal, 0, t1.length);
    System.arraycopy(t2, 0, rVal, t1.length, t2.length);
    System.out.println(Arrays.toString(rVal));
}
```

Κώδικας 6.22 *Συνένωση πινάκων*

Αφού δηλώσαμε τους πίνακες t1 και t2, δημιουργούμε τον πίνακα rVal με μήκος ικανό να φιλοξενήσει τα δεδομένα των t1 και t2. Στη συνέχεια, με την arraycopy μεταφέρουμε τα δεδομένα από τους t1 και t2. Τέλος, τυπώνουμε τον rVal ώστε να δείξουμε πως η συνένωση έγινε σωστά.

6.5.6 Εφαρμογή της arraycopy

Να γράψετε κώδικα που καταρχάς ορίζει τον πίνακα

```
char[][] copyFrom = {
     {'d', 'e'},
```

```
            {'c', 'a', 'f', 'f', 'e', 'i', 'n'},
            {'a', 't', 'e', 'd'}
        };
```

Στη συνέχεια, ορίστε έναν πίνακα, copyTo, ικανών διαστάσεων ώστε να μπορεί να φιλοξενήσει αντίγραφο του copyFrom. Στη θέση 2 του copyTo τοποθετήστε τους χαρακτήρες 'l', 'i', 'k', 'e'. Αντιγράψτε από τον copyFrom στον copyTo τους πίνακες στις θέσεις 0 και 1. Εμφανίστε τον copyTwo.

Λύση

```
    public static void copy() { ////Κώδικας 6.23
        char[][] copyFrom = {
            {'d', 'e'},
            {'c', 'a', 'f', 'f', 'e', 'i', 'n'},
            {'a', 't', 'e', 'd'}
        };
        char[][] copyTo = new char[3][];
        copyTo[2] = new char[]{'l', 'i', 'k', 'e'};
        System.arraycopy(copyFrom, 1, copyTo, 0, 2);
        for (char[] aElement : copyTo) {
            System.out.println(Arrays.toString(aElement));
        }
    }
```

Κώδικας 6.23 *Λύση της άσκησης 6*

6.5.7 Μεταβολή μεγέθους

Να υλοποιήσετε συνάρτηση στην οποία να δηλώνετε έναν μονοδιάστατο πίνακα ακέραιων 6 θέσεων, έστω t. Στη συνέχεια καταχωρήστε τους ακεραίους 1 έως 6 στον t. Ορίστε επίσης μια ακέραια μεταβλητή, έστω newSize. Μεταβάλλεται το μήκος του t σε newSize. Φροντίστε ώστε ο κώδικάς σας να λειτουργεί σωστά είτε το newSize είναι μεγαλύτερο από το 6 είτε μικρότερο. Τυπώστε τον t ώστε να διαπιστώσετε πως έχει το αναμενόμενο μήκος.

Λύση

```
1  static void resize() { //Κώδικας 6.24
2      int[] t={1,2,3,4,5,6};
3      int newSize=4;
4      int[] tmp=new int[newSize];
5      System.arraycopy(t, 0, tmp, 0, t.length<newSize?t.length:newSize);
6      t=tmp;
7      System.out.println(Arrays.toString(t));
8  }
```

Κώδικας 6.24 *Μεταβολή μεγέθους πίνακα*

Ορίζουμε τον t και την newSize σύμφωνα με την εκφώνηση. Στη συνέχεια δημιουργούμε έναν άλλο πίνακα με μήκος newSize. Αμέσως μετά αντιγράφουμε δεδομένα από τον t στον tmp. Προσοχή σε αυτό το σημείο. Αν το newSize είναι μεγαλύτερο ή ίσο από το t.length, τότε μπορούν να αντιγραφούν όλα τα δεδομένα του t. Αν όμως είναι μικρότερο, τότε θα πρέπει να αντιγραφούν μόνο όσα χωρούν. Στον κώδικα 6.24 ο έλεγχος αυτός γίνεται με τη βοήθεια του τριαδικού τελεστή στη γραμμή 5. Στη γραμμή 6 εκχωρούμε την αναφορά tmp στην t. Στη συνέχεια η t δείχνει σε πίνακα μεγέθους newSize.

6.6 Ασκήσεις προς λύση

1. Να γράψετε κώδικα στον οποίο να ορίζετε τους δισδιάστατους πίνακες ακέραιων 3x4, t1 και t2. Στη συνέχεια να δημιουργήσετε ένα πίνακα sum τέτοιον ώστε sum[i][j]=t1[i][j]+t2[i][j]. Εμφανίστε τον sum και ελέγξτε την ορθότητα του κώδικα.

2. Να γράψετε κώδικα που υπολογίζει ποια γραμμή δισδιάστατου πίνακα πραγματικών έχει το μεγαλύτερο άθροισμα στοιχείων.

3. Να γράψετε κώδικα που υπολογίζει ποια στήλη δισδιάστατου πίνακα πραγματικών έχει το μεγαλύτερο άθροισμα στοιχείων.

4. Να γράψετε κώδικα που αρχικοποιεί πίνακα ακέραιων 10x5 με τυχαίες τιμές από 0 έως 8. Τα δεδομένα σε κάθε γραμμή του πίνακα αναπαριστούν τις ώρες που εργάστηκε ένας εργαζόμενος κάθε μια από τις 5 εργάσιμες ημέρες μιας εβδομάδας. Υπολογίστε το σύνολο των ωρών που εργάστηκε κάθε εργαζόμενος κατά τη διάρκεια της εβδομάδας. Εμφανίστε τα σύνολα ταξινομημένα κατά φθίνουσα σειρά.

5. Να γράψετε κώδικα που αρχικοποιεί έναν δισδιάστατο πίνακα ακέραιων σε τυχαίες τιμές 0 και 1. Να υπολογίσετε πόσα 0 και πόσα 1 υπάρχουν σε κάθε γραμμή του πίνακα και πόσα σε όλον τον πίνακα.

6. Σε έναν δισδιάστατο πίνακα από String καταχωρήστε σε κάθε γραμμή του ένα κράτος της Ευρώπης με την πρωτεύουσά του. Στη συνέχεια υποβάλλετε 10 ερωτήσεις στον χρήστη και αξιολογήστε τις απαντήσεις του. Κάθε ερώτηση μπορεί να δίνει την ονομασία του κράτους και να ζητά την πρωτεύουσα ή να δίνει την πρωτεύουσα και να ζητά το κράτος.

7. Να αναπτυχθεί συνάρτηση που λαμβάνει ως παράμετρο έναν πίνακα ακέραιων και επιστρέφει 1 αν ο πίνακας είναι ταξινομημένος κατ’ αύξουσα σειρά, -1 εν είναι ταξινομημένος κατά φθίνουσα σειρά και 0 αν δεν είναι ταξινομημένος.

8. Να δηλώσετε έναν πίνακα, a, από συμβολοσειρές. Να εκχωρήσετε μια συμβολοσειρά σε κάθε θέση του πίνακα. Στη συνέχεια, να δημιουργήσετε ένα αντίγραφο του a τέτοιο ώστε κάθε αλλαγή σε μια συμβολοσειρά του a να μην επηρεάζει την αντίστοιχη συμβολοσειρά στο αντίγραφο.

Βιβλιογραφία

[1] C. Horstmann, Big Java: Early Objects, 7th Edition | Wiley, 7th ed. Laurie Rosatone, 2013. Accessed: Sep. 14, 2021. [Online]. Available: https://www.wiley.com/en-us/Big+Java%3A+Early+Objects%2C+7th+Edition-p-9781119499091

[2] D. Liang, Εισαγωγή στον Προγραμματισμό Java, 10η. Θεσσαλονίκη: Τζιόλας.

Κεφάλαιο 7

Σύνοψη
Σε αυτήν την ενότητα γίνεται εισαγωγή στις στατικές συναρτήσεις. Παρουσιάζονται οι τυπικές και πραγματικές παράμετροι καθώς και η τιμή επιστροφής. Εξηγείται η διαφορά μεταξύ παραμέτρων τιμής και αναφοράς. Αναλύεται η λίστα παραμέτρων μεταβλητού μήκους και οι παράμετροι της main. Επίσης, συζητούνται τρόποι αξιοποίησης στατικών μεταβλητών ενώ γίνονται οι απαραίτητες επισημάνσεις για την πρόκληση τυχόν απροσδόκητων λαθών κατά την κλήση συναρτήσεων. Επιπλέον, παρουσιάζονται μερικές χρήσιμες προκαθορισμένες συναρτήσεις, συζητούνται οι προσδιοριστές προσπέλασης και αναλύεται η χρήση των πακέτων.

Προαπαιτούμενη γνώση
Οι θεμελιώδεις τύποι και βασική γνώση του τύπου String. Βασική είσοδος και έξοδος. Η λειτουργία των τελεστών. Οι δομές επιλογής και οι επαναληπτικές δομές. Η διαχείριση των πινάκων.

Λέξεις κλειδιά
Στατικές συναρτήσεις, στατικές μεταβλητές, Υπερφόρτωση, τυπική παράμετρος, πραγματική παράμετρος, παράμετρος τιμής, παράμετρος αναφοράς, λίστα παραμέτρων μεταβλητού μήκους, τιμή επιστροφής.

7. Στατικές Συναρτήσεις και Μεταβλητές

Είναι συχνό φαινόμενο το ίδιο τμήμα κώδικα να είναι αναγκαίο σε διάφορα σημεία ενός προγράμματος. Ένας απλοϊκός τρόπος για να αντιμετωπιστεί η αναγκαιότητα αυτή είναι να επαναλάβουμε το ίδιο τμήμα όπου και όσες φορές αυτό χρειάζεται. Όμως, η προσέγγιση αυτή παρουσιάζει σοβαρά μειονεκτήματα. Καταρχάς, καθιστά τη συντήρηση του κώδικα πολύ δύσκολη. Αν χρειαστεί να κάνω μια βελτίωση ή οποιαδήποτε άλλη μεταβολή στο τμήμα του κώδικα που επαναλαμβάνεται, τότε θα πρέπει να βρω όλα τα σημεία στα οποία ενσωματώνεται και να τα μεταβάλω ένα προς ένα. Φανταστείτε τώρα πόσο δυσχεραίνεται η συντήρηση του κώδικα αν στο πρόγραμμά σας έχετε πολλά τμήματα κώδικα που το καθένα χρειάζεται σε πολλά σημεία, κάτι που είναι πολύ συνηθισμένο στα σύγχρονα προγράμματα.

Ένα άλλο θέμα μάλλον σημαντικότερο και από τη δυσκολία στη συντήρηση είναι η έλλειψη αφαιρετικότητας (abstraction). Ας υποθέσουμε πως έχουμε ένα τμήμα κώδικα που υπολογίζει τη δύναμη ενός αριθμού υψωμένου σε μια δύναμη. Έστω κατά την ανάπτυξη της εφαρμογής μας, χρειαζόμαστε αυτόν τον υπολογισμό. Πού είναι το τμήμα του κώδικα που τον κάνει; Θα πρέπει να ψάξουμε. Έστω ότι το βρήκαμε. Ναι, όμως εκείνο το τμήμα υπολογίζει την τιμή της μεταβλητής x υψωμένης στην τιμή της μεταβλητής k. Σε ένα άλλο σημείο χρειαζόμαστε τον ίδιο υπολογισμό αλλά για διαφορετικές μεταβλητές. Πρέπει λοιπόν να συντηρούμε σε πολλά σημεία όχι απλώς τον ίδιο κώδικα αλλά περίπου τον ίδιο. Οι μεταβολές όμως από σημείο σε σημείο ενδέχεται να εισάγουν λάθη.

Σίγουρα θα ήταν βολικό να κατασκευάζαμε ένα υποπρόγραμμα που θα είχε κατάλληλη ονομασία, π.χ. power ή calcPower έτσι ώστε όποτε χρειαζόμασταν να υπολογίσουμε τη δύναμη ενός αριθμού, να το καλούσαμε να έκανε τους υπολογισμούς και να μας ενημέρωνε με το αποτέλεσμα. Ένα τέτοιο υποπρόγραμμα βέβαια δεν θα έπρεπε να υπολογίζει κάθε φορά τη δύναμη του ίδιου αριθμού υψωμένου στον ίδιο εκθέτη. Επομένως θα έπρεπε να υπάρχει τρόπος κάθε φορά που το καλούμε να το ενημερώνουμε σχετικά με τη βάση και τον εκθέτη του απαιτούμενου υπολογισμού.

Για καλή μας τύχη, τέτοιου είδους υποπρογράμματα υποστηρίζονται στην Java και ονομάζονται συναρτήσεις. Οι παράμετροι των συναρτήσεων είναι ο μηχανισμός μέσω του οποίου δίνουμε στη συνάρτηση τα δεδομένα στα οποία θα επενεργήσει ενώ ο κύριος τρόπος που η συνάρτηση μας επιστρέφει την απάντησή της είναι η τιμή επιστροφής της συνάρτησης.

Με αυτόν τον τρόπο επιτυγχάνουμε σημαντική αφαιρετικότητα στα προγράμματά μας. Δεν είναι αναγκαίο να γνωρίζουμε τις λεπτομέρειες του κώδικα μιας συνάρτησης για να τη χρησιμοποιήσουμε. Αρκεί να γνωρίζουμε τι κάνει (και όχι πώς το κάνει), το όνομά της, τις παραμέτρους που δέχεται και τι αποτελέσματα επιστρέφει. Έτσι είναι εύκολο να χρησιμοποιηθούν συναρτήσεις και μάλιστα από ένα ευρύτερο κοινό και όχι μόνο από αυτούς που τις έχουν υλοποιήσει.

Πέραν της τιμής επιστροφής της, μια συνάρτηση είναι δυνατό να παράγει αποτελέσματα και με άλλους τρόπους. Για παράδειγμα, μία συνάρτηση εισάγει σε έναν πίνακα μια σειρά από τιμές και επιστρέφει τον αριθμό